

Real-Time Fleet & Delivery Management System

Project Overview

A real-time fleet and delivery management platform that tracks vehicles live, manages drivers and routes, and provides operational dashboards for admins. Built as a **collaborative full-stack project** following industry-style GitHub workflows.

Tech Stack

Backend

- Node.js
- Express.js
- WebSockets (Socket.IO)
- MongoDB (Mongoose)
- JWT Authentication

Frontend

- React (Vite)
- Tailwind CSS
- REST + WebSocket integration

DevOps / Tools

- Docker
 - GitHub (PR-based workflow)
 - Postman
-

Folder Structure (High Level)

```
root/  
├─ backend/  
│   └─ src/  
│       ├── controllers/  
│       ├── routes/  
│       ├── models/  
│       ├── services/  
│       ├── sockets/  
│       └─ app.js  
└─ package.json
```

```
|
|├ frontend/
|│├ src/
|│├ public/
|│└ package.json
|├ docs/
|│├ api-contracts.md
|│├ websocket-events.md
|│└ system-architecture.md
|├ .env.example
|├ CONTRIBUTING.md
|└ README.md
```

CONTRIBUTING.md

Contribution Guidelines

This project follows a **pull-request based workflow**.

Branch Rules

- `main` → stable / production-ready
- `dev` → integration branch
- `feature/*` → individual features

Workflow

1. Create a feature branch from `dev`
2. Commit with meaningful messages
3. Push and open a Pull Request to `dev`
4. At least **1 approval** required
5. Merge only via PR (no direct push to main)

Commit Message Convention

```
feat: add vehicle tracking API
fix: resolve socket reconnection bug
docs: update API contracts
```

.env.example

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/fleetdb
JWT_SECRET=your_jwt_secret
SOCKET_PORT=5001
```

docs/api-contracts.md

API Contracts

Auth

POST /api/auth/login

```
Request:
{ "email": "", "password": "" }
```

```
Response:
{ "token": "JWT_TOKEN" }
```

Vehicles

GET /api/vehicles – Get all vehicles

POST /api/vehicles – Add vehicle

```
{ "vehicleNumber": "", "driverId": "" }
```

docs/websocket-events.md

WebSocket Events

Client → Server

- join_vehicle_room
- location_update

Server → Client

- live_location
- vehicle_status

Payload Example:

```
{ "vehicleId": "", "lat": 0, "lng": 0 }
```

docs/system-architecture.md

System Architecture

- REST APIs handle CRUD operations
- WebSockets handle real-time location updates
- MongoDB stores vehicles, drivers, trips
- Frontend subscribes to live socket streams

Flow: Driver App → WebSocket → Backend → Admin Dashboard

How to Run

Backend

```
cd backend  
npm install  
npm run dev
```

Frontend

```
cd frontend  
npm install  
npm run dev
```

Team Collaboration

- 2 developers
- Code reviews via PRs

- Branch protection enabled
 - Tasks tracked via GitHub Issues
-



Future Enhancements

- Route optimization
- Role-based access control
- Analytics dashboard
- CI/CD with GitHub Actions